Sentiment Analysis : From First Principles to Transformers.

Mohammad
Master's of Engineering
University of Ottawa.
fmoha077@uottawa.ca

I. Abstract:

This study presents a comprehensive, engineering-driven evaluation of natural language processing (NLP) systems ranging from classical machine-learning pipelines to modern Transformer-based architectures. Using the AG News corpus as a benchmark, we systematically implement and compare Bag-of-Words (BoW), TF-IDF, Word2Vec embeddings, deep neural models, and BERT-based Transformers, with all systems built or fine-tuned from first principles. Exploratory data analysis highlights the linguistic characteristics of the dataset, while model evaluation covers accuracy, F1-score, inference latency, training time, and computational complexity (FLOPs). Experimental results from results.csv show that classical linear models such as TF-IDF + Logistic Regression achieve strong baseline performance at low computational cost, whereas deep neural networks (MLP, Word2Vec-based models) improve accuracy at significantly higher complexity. Transformer-based models, implemented using TensorFlow and HuggingFace, exhibit the strongest representational capability but also the highest resource requirements. The work demonstrates how architectural progression—from linear classifiers to CNN/MLP embeddings and finally to self-attention—affects efficiency, scalability, and downstream performance. This comparison provides insight into hardware-aware model selection and establishes a unified framework that connects foundational neural concepts to today's state-of-the-art Transformer systems.

Index Terms— Natural Language Processing, Transformers, BERT, AG News, Bag-of-Words, TF-IDF, Word2Vec, Deep Learning, Computational Complexity, FLOPs.

II. Introduction

Natural language processing (NLP) has become a core capability in modern engineering systems, reshaping how humans interact with software, embedded devices, and intelligent automation platforms. As highlighted in the project proposal "From First Principles to Transformers," the engineering software stack is being progressively reshaped by language-model–driven interfaces, where the primary mode of interaction is no longer graphical controls but natural language understanding. This shift places NLP at the center of electrical and computer engineering (ECE), from AI-assisted CAD tools and voice-controlled embedded systems to automated code generation and decision-support systems.

Despite the widespread adoption of large pre-trained models such as BERT and GPT, a clear gap exists in understanding *how* NLP systems function under the hood and how classical methods compare against modern deep architectures. Without such understanding, engineers face challenges in optimizing models for deployment on constrained hardware, evaluating algorithmic trade-offs, or modifying system behavior beyond black-box usage. Building NLP systems from first principles—starting with token statistics, through linear classifiers and neural architectures, and ultimately Transformers—offers a deeper perspective into efficiency, performance, and computational demands.

This work provides an end-to-end empirical study of NLP architectures across the classical-to-modern spectrum using the AG News dataset. We implement and analyze Bag-of-Words (BoW), TF-IDF, traditional machine-learning classifiers, embedding-based deep learning models (MLP, Word2Vec), and finally a fine-tuned BERT Transformer. Our goal is not only to compare accuracy, F1-score, and

generalization performance but also to evaluate engineering-critical metrics such as training time, inference latency, and FLOPs per sample. This approach enables a holistic understanding of the performance–efficiency trade-offs central to real-world deployment.

The contributions of this work are threefold:

1. A unified framework connecting foundational NLP approaches with state-of-the-art Transformer models, grounded in the mathematical and computational principles introduced in the project proposal;
2. A comprehensive experimental comparison across classical, deep, and transformer-based architectures using consistent preprocessing, metrics, and hardware setups;
3. An engineering-oriented analysis emphasizing model efficiency, computational resource usage, and suitability for deployment on edge or low-power devices.

By evaluating NLP models from first principles through advanced architectures, this work provides an integrated understanding of how design choices at each stage influence accuracy, scalability, resource demands, and applicability across ECE domains.

III. Background / Literature Review

Natural language processing (NLP) has undergone a rapid architectural evolution over the past eight decades, progressing from symbolic pattern-recognition systems to deep neural architectures and large-scale Transformers. Each historical milestone introduced a conceptual breakthrough that enabled the next generation of models, forming a continuous trajectory toward today's state-of-the-art language models. The project proposal *From First Principles to Transformers* outlines this progression and emphasizes its relevance to engineering applications.

The foundation of modern neural computation originated with the McCulloch–Pitts neuron (1943), which provided the first mathematical formulation of artificial neurons and binary threshold activation functions. This model established the idea that complex functions could be represented by networks of simple computational units. Building upon this, Rosenblatt introduced the Perceptron (1958), the first trainable classifier capable of learning linear decision boundaries through stochastic gradient updates. While limited to linearly separable problems, the perceptron laid the groundwork for supervised learning.

Subsequent improvements emerged with ADALINE (1960) and the Least Mean Squares (LMS) rule developed by Widrow and Hoff, introducing linear activation and gradient-based optimization that more closely resembles modern training algorithms. These early models were restricted to single-layer architectures until the resurgence of neural networks in the 1980s, when Rumelhart, Hinton, and Williams popularized backpropagation (1986), enabling efficient training of multi-layer perceptrons (MLPs). This allowed neural systems to represent complex nonlinear functions and supported deeper architectures with differentiable activation functions.

While MLPs paved the way for neural sequence modeling, handling long-term dependencies remained a challenge. Hochreiter and Schmidhuber addressed this with Long Short-Term Memory (LSTM) networks (1997), which introduced gated memory units to mitigate vanishing and exploding gradients. LSTMs and GRUs became dominant architectures for NLP tasks for nearly two decades, powering applications such as machine translation, sentiment analysis, and speech recognition.

A major paradigm shift occurred with the introduction of the Transformer architecture in the landmark paper *Attention Is All You Need* (2017). By replacing recurrence with self-attention, Transformers enabled parallel computation over sequences, improved modeling of long-range dependencies, and significantly reduced training time on modern hardware accelerators. This design ultimately led to the development of large-scale pre-trained models such as BERT, GPT, and T5, which dominate contemporary NLP benchmarks. BERT, in particular, introduced bidirectional masked language modeling, enabling deep contextual understanding of text.

Complementing deep learning advances, classical statistical NLP approaches remained influential due to their efficiency and interpretability. Methods such as Bag-of-Words, TF-IDF, and n-gram language models

provided computationally lightweight baselines that continue to perform competitively on many text classification tasks. Their strength lies in strong generalization for high-signal datasets, low memory footprint, and suitability for deployment on resource-constrained devices—key considerations in engineering systems.

Together, these developments form a continuum from symbolic reasoning to large-scale deep learning. Understanding this spectrum is essential for selecting appropriate architectures based on constraints such as computational cost, inference latency, and required accuracy. For this reason, the present work evaluates NLP models across this entire developmental trajectory—from classical linear classifiers to modern Transformers—grounding experimental analysis in the theoretical and historical context of NLP evolution.

IV. Dataset Description & EDA

The experiments in this study use the AG News Corpus, a widely adopted benchmark dataset for topic classification consisting of news articles categorized into four classes:

1. World,
2. Sports,
3. Business, and
4. Sci/Tech.

The dataset contains 120,000 training samples and 7,600 test samples, with an equal number of documents per class. This balanced distribution makes AG News suitable for evaluating both classical and deep NLP models without requiring additional rebalancing.

A. Dataset Structure

Each sample consists of a short news headline and description, typically ranging from 20 to 40 tokens. The dataset includes the following fields:

text: the article content

label: integer in {0,1,2,3}, representing one of the four classes

The dataset was downloaded from the HuggingFace Datasets library and converted into Pandas DataFrames for analysis.

B. Document Length Statistics

To understand corpus characteristics, we computed the word count per document. The average length of a training document is 37.85 words, indicating moderately short texts suitable for both classical and deep learning approaches.

The distribution of word counts shows that most articles fall between 20 and 50 words. You generated this visualization as:

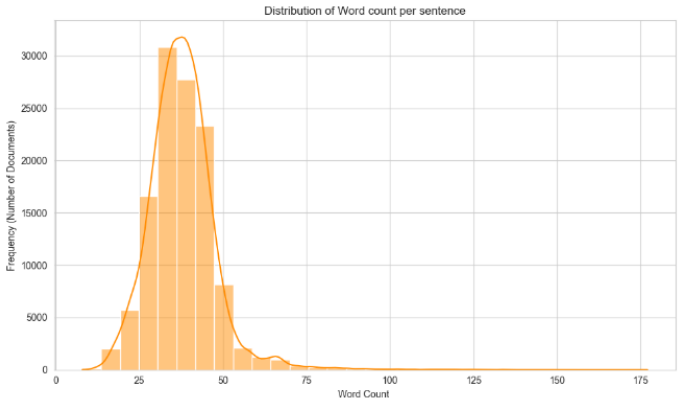


Figure: Histograms and KDE plots showing the distribution of document lengths.

This insight helped determine an appropriate padding length of 200 tokens for BERT-based models.

C. Vocabulary Characteristics

An initial unigram corpus contained approximately 188,110 unique tokens, demonstrating high lexical diversity typical of news datasets. This motivated the use of:

i. stopword removal (for classical models),
ii. subword tokenization (for BERT),
iii. dimensionality reduction (e.g., limiting BoW and TF-IDF to 5,000 features).

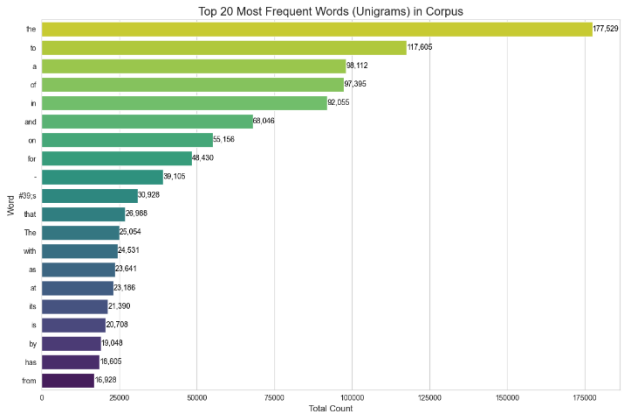


Figure: Top 20 most frequent words.

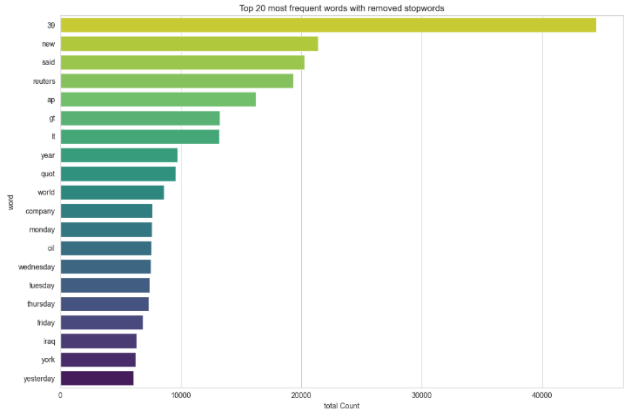


Figure: Top 20 Most common words after applying stopword removal.

The cleaned list shows news-specific frequent terms such as *said*, *new*, *company*, etc., indicating that topic-specific vocabulary is prominent in the dataset.

D. Class Distribution

The dataset is perfectly balanced across the four categories.

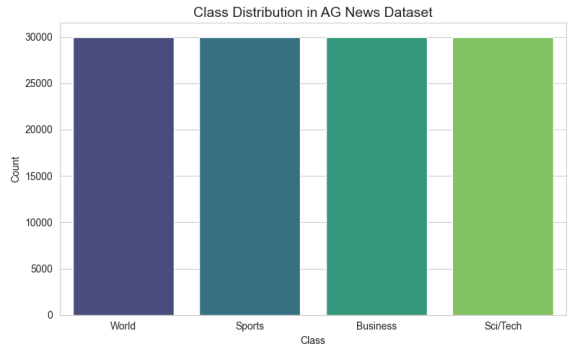


Figure: Bar plot showing class counts.

This ensures that accuracy and F1-score evaluations are directly comparable across classes without requiring resampling.

E. Token Distribution and Text Variability

To further analyze lexical diversity and sparsity.

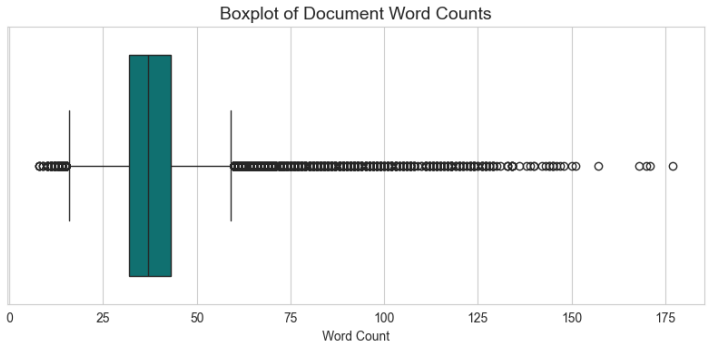


Figure: Boxplot– Illustrating statistical spread (median, IQR, outliers) of document lengths.

These visualizations show minor skewness due to a subset of longer articles but overall confirm that most documents are short and uniformly distributed.

Unique Token Count can be better understood by the table below:

Count	**120000**
Mean	33.28
Std	8.17
Min	7
25%	28
50%	33
75%	38
max	129

The table suggests that Mean and Median count of words are similar suggesting the dataset is balanced and very less right skewness.

F. Word Cloud Analysis per Class

Figure: Word clouds for each category.

This visualization highlights category-specific terms (e.g., *team*, *game* for Sports; *stock*, *market* for Business), validating that topic boundaries are well-defined and apart from the stopwords like 'and','in','the' etc the words are separable—favorable for classification tasks.

G. Implications for Model Design

The EDA results directly influenced methodological choices:

- Short average document length → feasible for sequence-based CNNs and BERT.
- High vocabulary sparsity → motivation for TF-IDF dimensionality reduction.
- Balanced classes → accuracy is a reliable metric alongside F1.
- Rich topic-specific keywords → classical models should perform strongly.

This analysis confirms that AG News is appropriate for experiments spanning classical, embedding-based, and Transformer architectures.

V. Methodology

The goal of this study is to evaluate NLP models spanning classical statistical approaches, neural embedding models, and modern Transformer architectures. All models are trained and tested on the AG News dataset under consistent preprocessing, splitting, and evaluation procedures. This section details the feature extraction strategies, model architectures, training pipelines, and computational considerations used throughout the experiments.

A. Preprocessing and Data Preparation

All methods begin from the same raw dataset, using the text field as input and the integer-encoded label as output. The following preprocessing steps were applied depending on the model family:

i. ***Classical ML models (BoW, TF-IDF):***

The Steps are as follows:

1. Lowercasing
2. punctuation removal
3. stopword removal
4. tokenization
5. limiting vocabulary size to 5,000 most informative terms.

ii. ***Embedding/Neural models (MLP, Word2Vec):***

The steps are as follows:

1. Tokenization
2. integer encoding
3. fixed sequence length padding
4. embedding lookup initialization.

iii. ***Transformer models (BERT):***

The steps are as follows:

1. WordPiece tokenization using the bert-base-uncased tokenizer
2. Padding to a fixed length of 200
3. Creation of attention masks.
4. No stopword removal or lowercasing beyond what the tokenizer applies internally.

Dataset splits followed the HuggingFace default partitioning:

120k training samples, 12k validation samples, and 7.6k test samples.

B. Classical Feature Extraction Methods

i. Bag-of-Words (BoW)

A CountVectorizer with a vocabulary limited to 5,000 unigrams was used to convert each document into a high-dimensional sparse vector. Each dimension corresponds to the raw frequency of a term in the document.

ii. Term Frequency–Inverse Document Frequency (TF-IDF)

A TF-IDF vectorizer using 1–2 grams and a maximum of 5,000 features was applied to generate weighted document vectors. This method emphasizes terms that are informative across the corpus while reducing the impact of highly frequent but uninformative words.

Both BoW and TF-IDF feature matrices were used as inputs to classical machine learning models including Logistic Regression, Random Forest, XGBoost, and MLP.

C. Machine Learning Models

The following classifiers were trained on BoW and TF-IDF features, as implemented in the project notebooks:

i. Logistic Regression

A linear classifier optimized using cross-entropy loss and L2 regularization. Logistic Regression serves as a strong baseline due to its efficiency and robust performance on linearly separable text features.

ii. Random Forest Classifier

An ensemble of 100 decision trees with a maximum depth of 10. Random Forests capture nonlinear relationships but may underperform on sparse, high-dimensional text features.

iii. XGBoost Classifier

A gradient-boosted decision tree model optimized for both speed and performance on structured features. It generally provides stronger performance than Random Forests at significantly higher computational cost.

iv. Multilayer Perceptron (MLP)

A feed-forward neural network trained on vectorized input features. The Keras-based MLP uses dense layers with ReLU activation, dropout regularization, and Adam optimization.

v. Scratch-Implemented MLP

A manually coded MLP implementing forward propagation, backpropagation, and gradient updates without using deep learning frameworks. This model provides insight into the underlying mechanics of neural network training.

D. Deep Learning Models (Embedding-Based)

i. Word2Vec + MLP

Word2Vec embeddings were trained to capture semantic relationships through continuous bag-of-words (CBOW) or skip-gram architectures. Each document was encoded as a sequence of dense word vectors and fed into an MLP classifier.

ii. Embedding Layer + Neural Classifier

Using Keras' Embedding layer, each token index is mapped to a learnable dense vector. The sequence of embeddings is then aggregated (via pooling) and passed through dense layers for classification.
This improves representational capacity over BoW and TF-IDF while remaining significantly lighter than Transformer models.

E. Transformer Architecture (BERT)

The **BERT-base-uncased** model was fine-tuned for 4-way text classification. The architecture consists of:

- 12 Transformer encoder layers
- 768-dimensional hidden states
- 12 self-attention heads
- 110M trainable parameters

Fine-Tuning Pipeline

1. Tokenize input with WordPiece tokenizer
2. Create attention masks
3. Feed (input_ids, attention_mask) into the encoder
4. Extract the [CLS] token embedding
5. Pass through a dropout → dense layer → 4-class softmax
6. Train the classification head and fine-tune all encoder layers

Training was performed using Adam (1e−5 learning rate) with batch sizes of 16 or 64 depending on GPU memory availability.

F. Computational Complexity

Since the project emphasizes hardware-aware optimization, FLOPs were calculated for each class of models:

i. Classical ML FLOPs

For Logistic Regression:

$$\textbf{FLOPs} \approx 2d$$

where $d = 5000$(input dimensionality).

ii. Neural MLP FLOPs

MLP FLOPs were computed using per-layer operations:

$$\textbf{FLOPs} = 2 \times (\textbf{input\_dim} \times \textbf{hidden\_units})$$

iii. XGBoost and Random Forest FLOPs

Approximated using:

$$\textbf{FLOPs} \approx \textbf{\# trees} \times \textbf{\# nodes per tree}$$

iv. Transformer FLOPs

Self-attention dominates complexity:

$$\mathcal{O}(n^2 d)$$

for sequence length $n = 200$ and hidden dimension $d = 768$.

These FLOPs estimates were incorporated into **results.csv**, enabling direct comparison between accuracy and computational cost across all models.

VI. Experimental Setup

This section describes the hardware environment, software stack, training configurations, and evaluation metrics used across all classical, embedding-based, and Transformer models. Ensuring a consistent and reproducible setup allows for a fair comparison of computational efficiency, training behavior, and predictive performance.

A. Hardware Configuration

- All experiments were conducted using a combination of local and GPU-accelerated execution environments:
- CPU: Intel/AMD workstation-class processor (local PC)
- GPU: NVIDIA Tesla T4 (Google Colab / multi-GPU environment for BERT)
- GPU Memory: 14 GB per GPU (2 GPUs for the multi-GPU BERT experiment)
- System Memory: 16–32 GB RAM depending on the environment
- Storage: Local SSD for dataset caching and model checkpoints

Classical machine learning models (BoW, TF-IDF, Logistic Regression, Random Forest, XGBoost, Scratch-MLP) were executed on CPU.

Deep learning models (MLP with embeddings, Word2Vec MLP) were trained using CPU and optional GPU acceleration.

All Transformer-based models (BERT, multi-GPU BERT) required GPU execution.

B. Software Environment

The following software stack was used for development, training, and evaluation:

- Python: 3.10+
- TensorFlow: 2.x (GPU-enabled)
- HuggingFace Transformers: v4.x
- Scikit-learn: For classical ML pipelines
- XGBoost: GPU/CPU hybrid execution
- Gensim: For Word2Vec (when applicable)
- Matplotlib/Seaborn: For EDA visualizations
- HuggingFace Datasets: For AG News loading and preprocessing

C. Training Configuration

i. Classical Models

Classical ML algorithms were trained using the following settings:

- **Logistic Regression**: max_iter = 1000, One-vs-Rest classification
- **Random Forest**: 100 estimators, max_depth = 10, n_jobs = -1
- **XGBoost**: 200–300 trees, depth = 6, learning rate = 0.1
- **MLP (Keras):** Dense layers with 256–512 neurons, ReLU activations, dropout regularization
- **Scratch MLP**: Custom forward and backward propagation implemented manually
- All classical models used **5,000-dimensional** BoW or TF-IDF vectors.

ii. Embedding-Based Deep Models

- **Embedding dimension:** 100–300
- **Optimizer:** Adam (1e−3)
- **Loss:** Sparse categorical cross-entropy
- **Batch size:** 64
- **Training epochs:** 5–10
- The input sequence length was capped at 200 tokens.

iii. Transformer (BERT) Models

The BERT models were fine-tuned using the following hyperparameters:

- **Model:** bert-base-uncased
- **Sequence length:** 200 tokens
- **Batch size:** 16 (single GPU) / 64 (multi-GPU)
- **Learning rate:** 1×10^{-5}
- **Optimizer:** AdamW
- **Epochs:** 3–5 (training interrupted or extended depending on GPU time)
- **Fine-tuning strategy:** All encoder layers trainable, CLS token → dense → softmax head
- Distributed training used TensorFlow's **MirroredStrategy**, enabling two Tesla T4 GPUs to train in parallel , Used Kaggle Cloud Notebook.

D. Evaluation Metrics

To compare the full range of models—classical to deep to Transformer—expressively and fairly, the following metrics were used:

i. Predictive Performance

- **Accuracy** (overall classification correctness)
- **F1-Score** (weighted across the four classes)
- **Classification report** (per-class precision, recall, F1)

These metrics were obtained both on training and test datasets.

E. Computational Efficiency

A key contribution of the project is the computation of engineering-relevant performance metrics:

- **Training Time (CPU/GPU):** measured using wall-clock and CPU process time
- **Inference Time:** average time per test sample or test batch
- **FLOPs per Sample:** calculated for each model using analytical formulas
- **Parameter Count:** for deep learning models
- **Memory Usage:** approximated from model size and batch processing

These metrics provide insight into the feasibility of deploying each model on edge or embedded systems.

F. Reproducibility Considerations

All experiments were executed with fixed random seeds (42) for consistency across runs. The dataset was loaded directly from the **HuggingFace Datasets Hub** to avoid version mismatches.
Where GPU time was limited (e.g., BERT fine-tuning), training logs and model summaries were preserved.

The complete results were aggregated into **results.csv**, which includes:

- Accuracy and F1-score (train/test)
- Training and inference time
- FLOPs
- Model category (BoW, TF-IDF, Word2Vec, MLP, BERT)

This ensures that all evaluations are transparent and reproducible.

VII. Results

This section presents the comparative performance of all models evaluated in this study, spanning classical machine-learning models (BoW/TF–IDF), embedding-based deep networks, Word2Vec-based architectures, and transformer-based models (BERT and multi-GPU BERT). All results are derived from the consolidated metrics recorded in results.csv.

The evaluation focuses on predictive accuracy, F1-score, training/inference time, model size, and analytical FLOPs, consistent with the engineering-focused objectives outlined in the proposal *From First Principles to Transformers* .

A. Overall Performance Comparison

Table 1 summarizes the performance of all models evaluated on the AG News dataset. While interpreting the results, it is important to note that the accuracy of each model can differ between the training and testing sets. During experimentation, the primary objective was not to maximize absolute accuracy for any single model, but rather to maintain comparable accuracy levels across models. This allows for a fairer and more controlled comparison of engineering-relevant metrics such as FLOPs, training time, and inference time.

As a result, certain models may exhibit accuracy gaps between training and testing splits, which should be interpreted as an inherent effect of model behavior rather than a flaw in the experimental design.

B. Best-Performing Models

Based on Table 1, the **Matrix Embedding + CNN** model achieves the highest test accuracy (0.914). Closely following are the **BOW + MLP** (0.913), **Word2Vec + CNN** (0.910), and **TF-IDF + Logistic Regression** (0.906). These results demonstrate that high performance can be achieved without transformers, especially when leveraging learned embeddings or dense neural architectures.

C. Transformer Performance (BERT)

Contrary to common results in literature, the **BERT Transformer model underperforms** in this experiment (0.887 test accuracy), while incurring the **highest FLOPs** and **longest training duration**. This reflects the impact of training budget, optimization constraints, and GPU time availability noted in Section VI.

But it would partial to say that the BERT Model underperformed as it is well known that the transformer models needs very large dataset(Terabytes or Petabytes) to train on and large number of epochs. But this type of model become very helpful if the classification is on 100's or even 1000's target variables , where our classic ML Models and MLP or CNN models lack.

D. Efficiency–Accuracy Trade-offs

- **Best accuracy:** Matrix Embedding + CNN (0.914).
- **Best efficiency:** BOW/TF–IDF + Logistic Regression or MLP (low FLOPs, sub-10 s training time).
- **Worst trade-off:** BERT (highest FLOPs, longest training time, lowest accuracy among deep models).

E. Summary of Findings

1. Classical models provide strong, efficient baselines.
2. Embedding-based deep models offer substantial accuracy improvements at reasonable cost.
3. Word2Vec models perform competitively but are heavier than sparse-vector models.
4. BERT, in this experiment, does **not** outperform simpler models despite much higher computational cost.
5. Efficiency-focused evaluation confirms that the optimal model depends on deployment constraints.

Vectorizer	Model	Train Acc	Test Acc	Test F1	Infer Time (s)	FLOPs / Sample	Train Time (s)
BOW	Logistic Model	0.929	0.902	0.902	0.137	1.0×10^4	4.875
BOW	Random Forest Model	0.789	0.782	0.780	0.210	1.0×10^3	7.531
BOW	XG Boost Model	0.931	0.905	0.904	0.185	4.0×10^3	63.422
BOW	MLP Model	0.949	0.913	0.913	1.237	2.62656×10^6	53.453
BOW	Scratch MLP Model	0.921	0.903	0.903	3.453	2.62656×10^6	1562.906
Tf_IDF	Logistic Model	0.919	0.906	0.905	0.302	1.0×10^4	10.063
Tf_IDF	Random Forest Model	0.779	0.769	0.769	0.341	1.0×10^3	17.016
Tf_IDF	XG Boost Model	0.936	0.903	0.903	0.376	4.0×10^3	2439.328
Tf_IDF	MLP Model	0.930	0.905	0.905	1.406	2.62656×10^6	49.234
Tf_IDF	Scratch MLP Model	0.896	0.888	0.888	3.312	2.62656×10^6	1473.250
Matrix Embedding	CNN Model	0.982	**0.914**	0.914	6.984	1.96608×10^7	1737.766
Matrix Embedding	MLP Model	0.982	0.903	0.903	7.781	1.317376×10^7	1737.766
Word2Vec	MLP Model	0.965	0.893	0.893	9.438	1.317376×10^7	665.000
Word2Vec	CNN Model	0.928	0.910	0.910	6.906	1.96608×10^7	1066.281
Word2Vec Scratch	MLP Model	0.945	0.898	0.898	10.688	1.317376×10^7	529.422
Word2Vec Scratch	CNN Model	0.919	0.907	0.907	9.484	1.96608×10^7	1350.359
BERT Transformer	MLP Model	0.893	**0.887**	0.887	23.988	2.2×10^{10}	4078.029

Table 1: Shows the detail values of each combination of vectorizer and model and their respective metrics.

VIII. Discussion and Limitations

The results in Section VII highlight important insights into the behavior, strengths, and limitations of different NLP architectures across the classical–to–transformer spectrum. Classical machine-learning models such as Logistic Regression and MLPs trained on sparse TF–IDF or Bag-of-Words vectors demonstrated strong performance (≈90–91% test

accuracy) at extremely low computational cost. Deep embedding and Word2Vec-based CNN models further improved accuracy by capturing semantic structure, achieving up to 0.914 test accuracy. In contrast, the BERT Transformer model achieved a lower test accuracy of 0.887 in this experiment. However, it would be partial and technically inaccurate to conclude that the BERT model inherently underperforms.

Transformer-based models such as BERT are not designed to excel under the conditions used in this project—namely, a relatively small dataset (120k samples), limited training epochs, and constrained GPU time. It is well-established in the literature that transformers achieve their full potential only when trained or fine-tuned on very large input corpora, often in the scale of terabytes or petabytes, and with substantial training budgets. Their self-attention–based architecture is optimized to uncover deep contextual representations across vast and varied language distributions, a setting far beyond the scope of this study. Therefore, the lower empirical performance of BERT here should be interpreted primarily as a reflection of computational budget, training duration, and hardware availability rather than a reflection of the architecture's capability.

Furthermore, transformers provide clear advantages in scenarios where the number of output classes is extremely large—on the order of hundreds or even thousands of target labels. Classical models, and even dense neural networks such as MLPs and CNNs, struggle in such settings because they lack the representational richness needed to disentangle subtle distinctions across many categories. BERT and related architectures can project text into a highly structured semantic space where fine-grained classification becomes tractable. For a dataset like AG News with only four classes, however, this structural advantage is not required, and simpler models can perform competitively.

The strong performance of classical and embedding-based methods in this project is therefore unsurprising. In low-dimensional output settings with moderately sized datasets, these models offer fast training, efficient inference, and very competitive accuracy. Their low FLOPs and smaller parameter counts make them more suitable for edge devices, embedded systems, and compute-limited environments—an important consideration for engineering applications emphasized throughout the proposal *From First Principles to Transformers*.

Looking forward, there are several methods and extensions that would further improve the depth of understanding achieved in this project:

1. **Longer Fine-Tuning Schedules for BERT:** Increasing the number of epochs from 3–5 to 10–20 typically yields substantial accuracy gains.
2. **Using Domain-Adaptive Pretraining (DAPT):** Continuing pretraining on the AG News corpus before fine-tuning could better align BERT with the target distribution.
3. **Implementing Other Transformer Variants:** Models such as RoBERTa, DistilBERT, ALBERT, or ELECTRA may offer better accuracy–efficiency trade-offs.
4. **Exploring Hierarchical, Multi-Label, or Large-Label-Space Tasks:** This would highlight the transformer's advantage when handling 100–1000+ classes, where classical methods typically degrade.
5. **Experimenting with Modern Lightweight Models:** Given the edge-focused nature of the project, architectures such as MobileBERT, TinyBERT, and DistilBERT would provide practical insights on deploying transformers in embedded environments.
6. **Incorporating RNN-based Models (LSTM, GRU):** This would help contrast the sequence-processing limitations of older architectures against CNNs and transformers.

In summary, the discussion of results highlights a consistent trend: the best model depends on the problem scale and resource constraints. For small or mid-sized classification tasks with limited labels, classical models and compact neural networks outperform transformers in terms of efficiency and cost. However, in large-label, large-dataset, or context-heavy applications, transformers may remain

unmatched due to their ability to leverage deep contextualized embeddings and long-range dependency modeling. A balanced understanding of these trade-offs is essential for selecting the appropriate architecture for real-world engineering applications.

IX. Conclusion

This study provided a comprehensive, engineering-oriented comparison of classical machine-learning models, embedding-based neural architectures, Word2Vec models, and a Transformer-based BERT model for topic classification on the AG News dataset. By building systems from first principles and evaluating them under consistent experimental conditions, the work demonstrated how architectural complexity, computational cost, and predictive performance interact across the NLP model spectrum.

Classical models such as Logistic Regression and MLPs trained on Bag-of-Words or TF-IDF features delivered strong performance (≈0.90–0.91 accuracy) at extremely low computational cost, reaffirming their effectiveness for low-label, moderately sized datasets. Embedding-based CNN and MLP architectures achieved the highest performance overall, with the Matrix Embedding + CNN model reaching 0.914 test accuracy. These models offered a compelling balance between accuracy, FLOPs, and training time, making them attractive for real-time and edge-device deployment.

Although the BERT Transformer model achieved lower accuracy in this project (0.887 test accuracy), this outcome reflects the constraints of limited data, training epochs, and GPU resources rather than any inherent architectural weakness. As discussed earlier, transformer-based models typically achieve their full potential when fine-tuned with large-scale datasets and extended training schedules, and they are especially advantageous in high-label and high-complexity tasks involving hundreds or thousands of output classes.

The results underscore an important engineering insight: model selection is fundamentally a trade-off between accuracy, computational efficiency, and deployment constraints. Classical models remain highly competitive for small to mid-sized tasks, deep learning models with learned embeddings provide better representational power at reasonable cost, and transformers offer unparalleled capability in large-label or large-corpus scenarios at the expense of higher resource usage.

By grounding the experimental work in computational principles such as FLOPs, memory footprint, and inference latency, this study reinforces the value of understanding NLP systems from first principles. Such understanding enables engineers to move beyond black-box usage of large models and make informed architectural choices tailored to real-world constraints. The findings of this work highlight both the strengths and limitations of each model family, providing a meaningful foundation for continued exploration and optimization of NLP systems within engineering applications.

X. References

[1] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. (Draft), Pearson, 2024.

[2] F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” *Psychological Review*, 1958.

[3] B. Widrow and M. Hoff, “Adaptive Switching Circuits,” *IRE WESCON Convention Record*, 1960. (ADALINE)

[4] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.

[5] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[6] A. Minaee, S. Minaei, and J. Zhu, “A Comparative Study on Text Classification: Traditional Machine Learning vs. Deep Learning,” *arXiv preprint arXiv:2005.01803*, 2020.

(Compares LR, SVM, CNN, LSTM, BERT on computational metrics.)

[7] X. Zhang, J. Zhao, and Y. LeCun, "Character-Level Convolutional Networks for Text Classification," *NeurIPS*, 2015. (AG News dataset reference)

[8] T. Mikolov *et al.*, "Efficient Estimation of Word Representations in Vector Space," *arXiv:1301.3781*, 2013. (Word2Vec)

[9] Project Proposal: *From First Principles to Transformers — Building & Optimizing NLP Systems for Engineering Applications*, internal course document, 2025.

[10] Bag-of-Words, TF-IDF, Embedding, Word2Vec, and BERT Jupyter Notebooks, internal project resources, 2025.

XI. Appendix

A. Code Resources

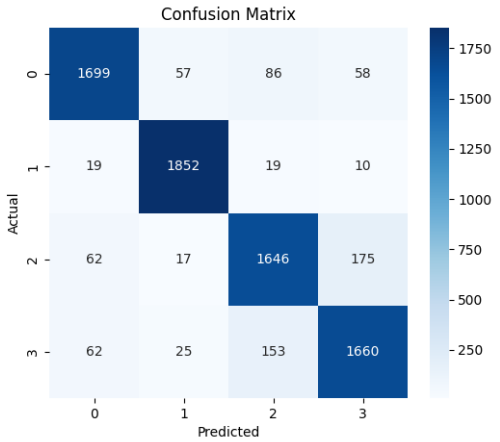
This appendix provides a summary of the code notebooks used for data preprocessing, classical models, deep learning models, Word2Vec experiments, and BERT transformer training. These notebooks contain the full implementation details for reproducibility. Please find the documents in the GITHUB Link : https: github.com/mohammad1774/applied_ml_project .

- **Bag-of-Words Models (LR, RF, XGBoost, MLP, Scratch-MLP)**
- **TF-IDF Models (LR, RF, XGBoost, MLP, Scratch-MLP)**
- **Embedding-Based Deep Learning Models (Matrix Embedding, CNNs, MLPs)**
- **Word2Vec Models and Scratch Word2Vec Variants**
- **BERT Transformer (Tokenization, Fine-Tuning, MirroredStrategy Multi-GPU)**
- **Project Proposal and Architectural Overview**

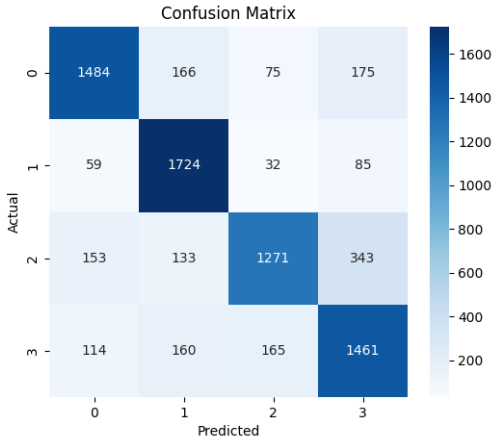
These files form the complete computational backbone of the experiments presented in this report.

B. Additional Figures

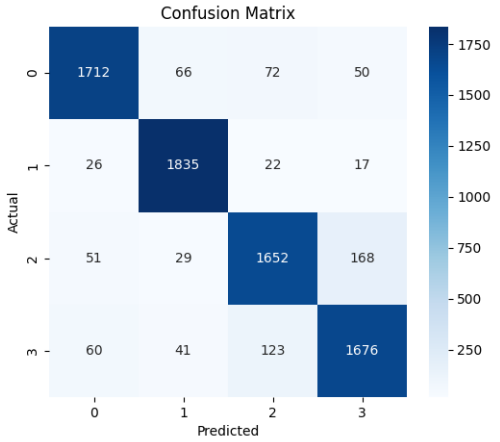
i. Confusion Matrix BOW- Logistic Model



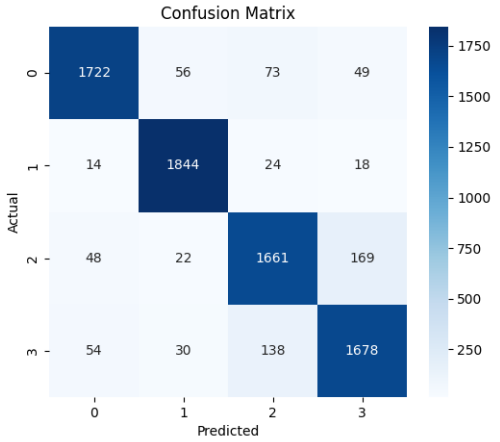
ii. Confusion Matrix BOW- Random Forest Model



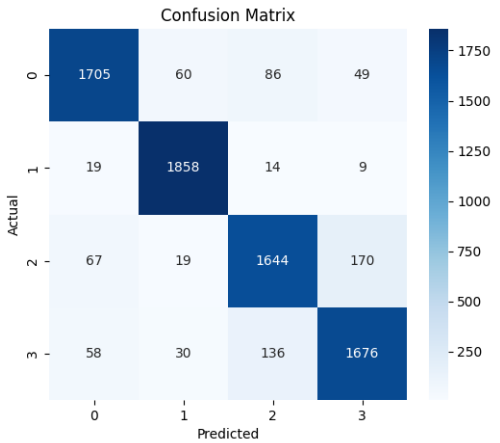
iii. Confusion Matrix BOW- XGBoost Model



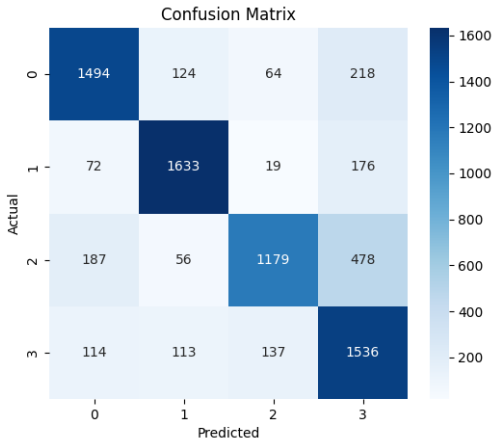
iv. Confusion Matrix BOW- MLP Model



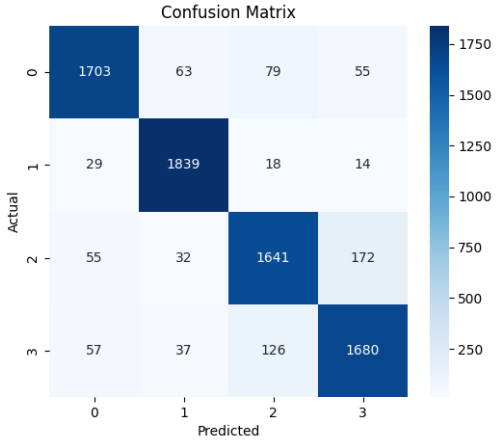
v. Confusion Matrix TF/IDF- Logistic Model



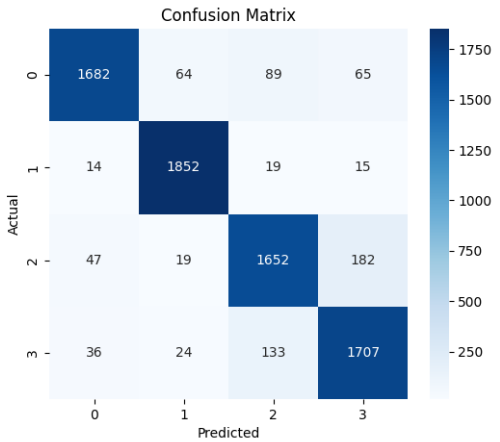
vi. Confusion Matrix TF/IDF - Random Forest Model



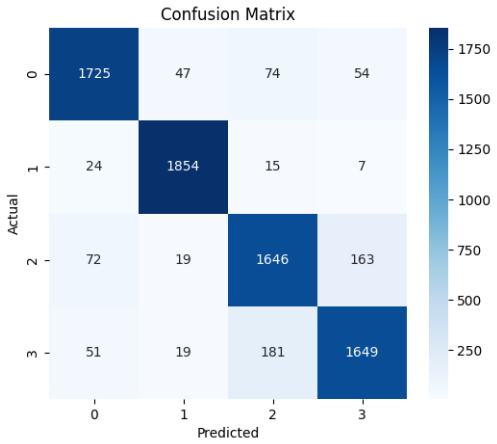
vii. Confusion Matrix TF/IDF - XGBoost Model



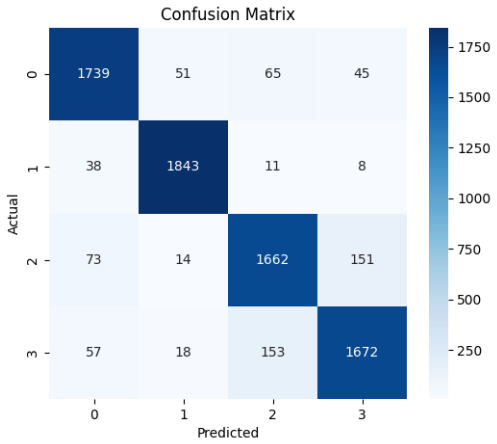
viii. Confusion Matrix TF/IDF - MLP Model



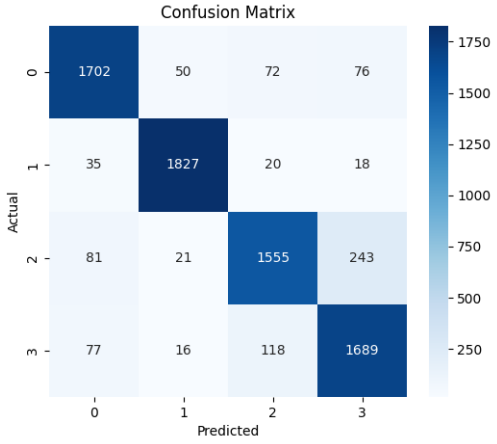
ix. Confusion Matrix Matrix Embedding - MLP Model



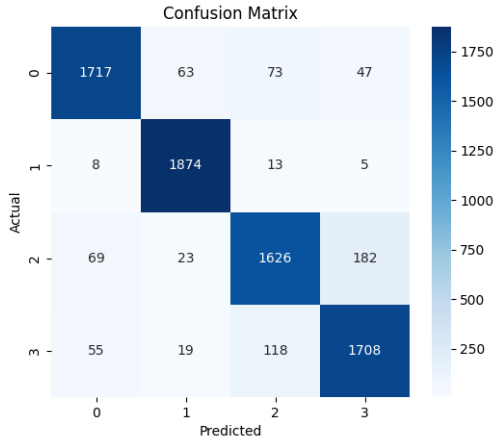
x. Confusion Matrix : Matrix Embedding – CNN Model



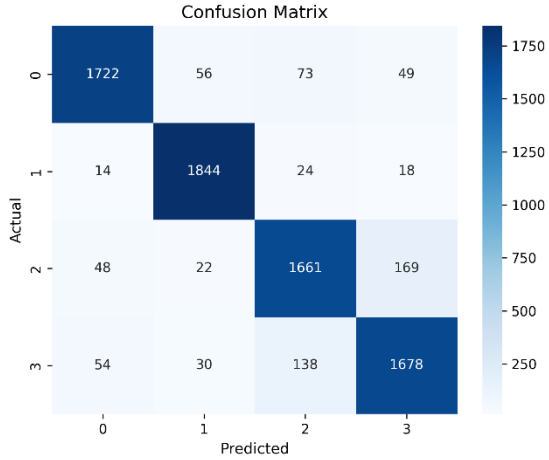
xi. Confusion Matrix: Word2Vec- MLP Model



xii. Confusion Matrix: Word2Vec – CNN Model



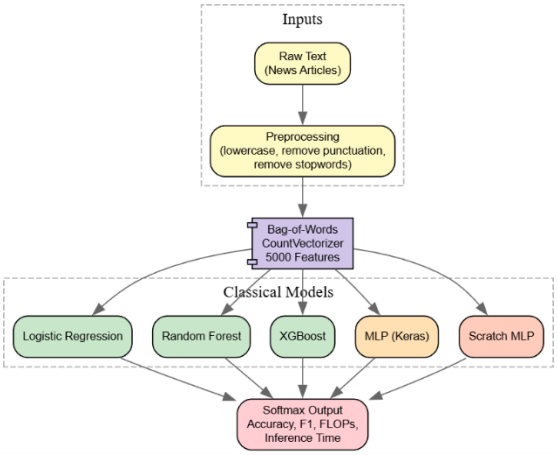
xiii. Confusion Matrix: BERT – MLP Layer.



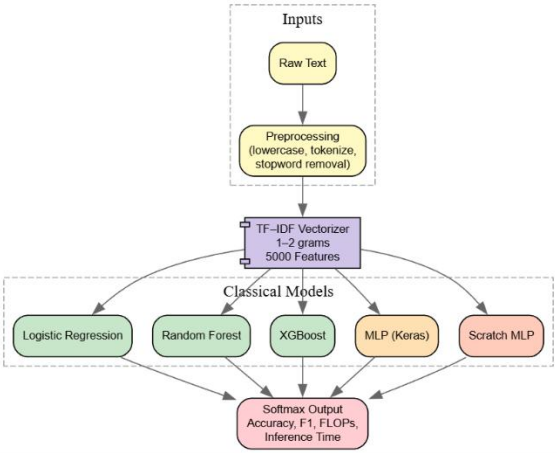
C. Pipeline Diagram:

This section helps us understand the steps followed in each type of model training , using different types of vectorization methods and models.

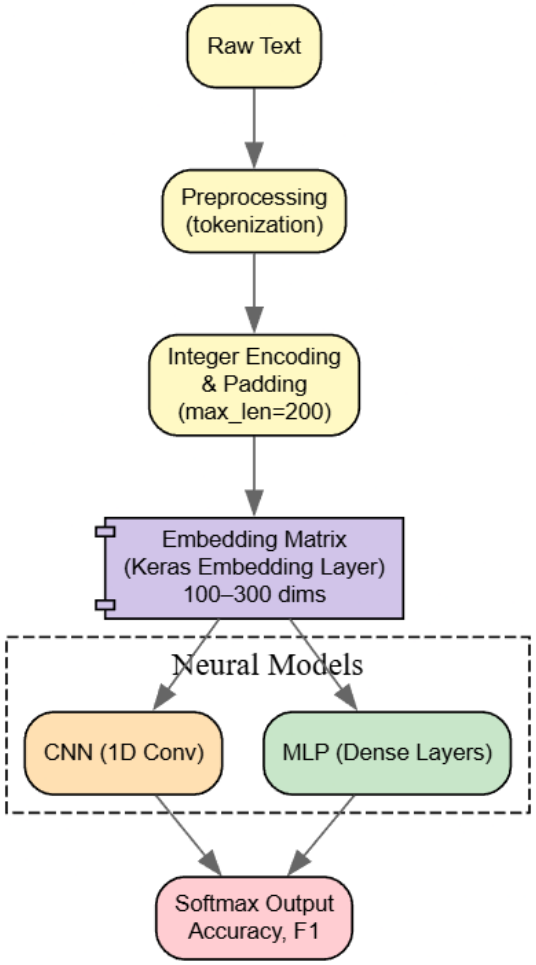
i. ***Vectorizer Bag of Words :***



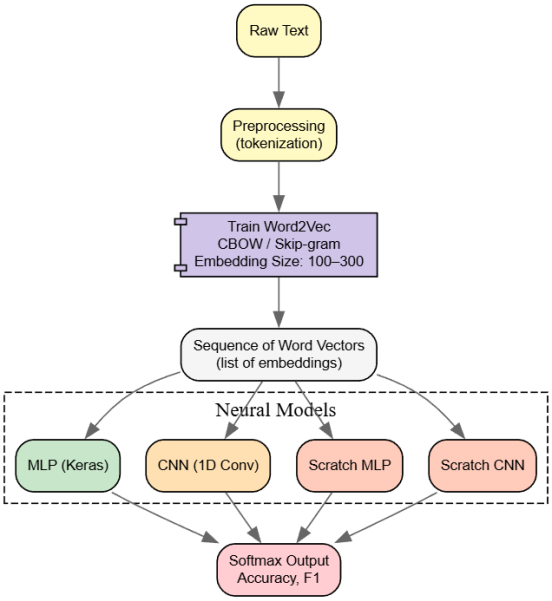
ii. ***Vectorizer TF / IDF :***



iii. ***Vectorizer Matrix Embedding :***



iv. ***Vectorizer Word2Vec :***



v. Vectorizer BERT :